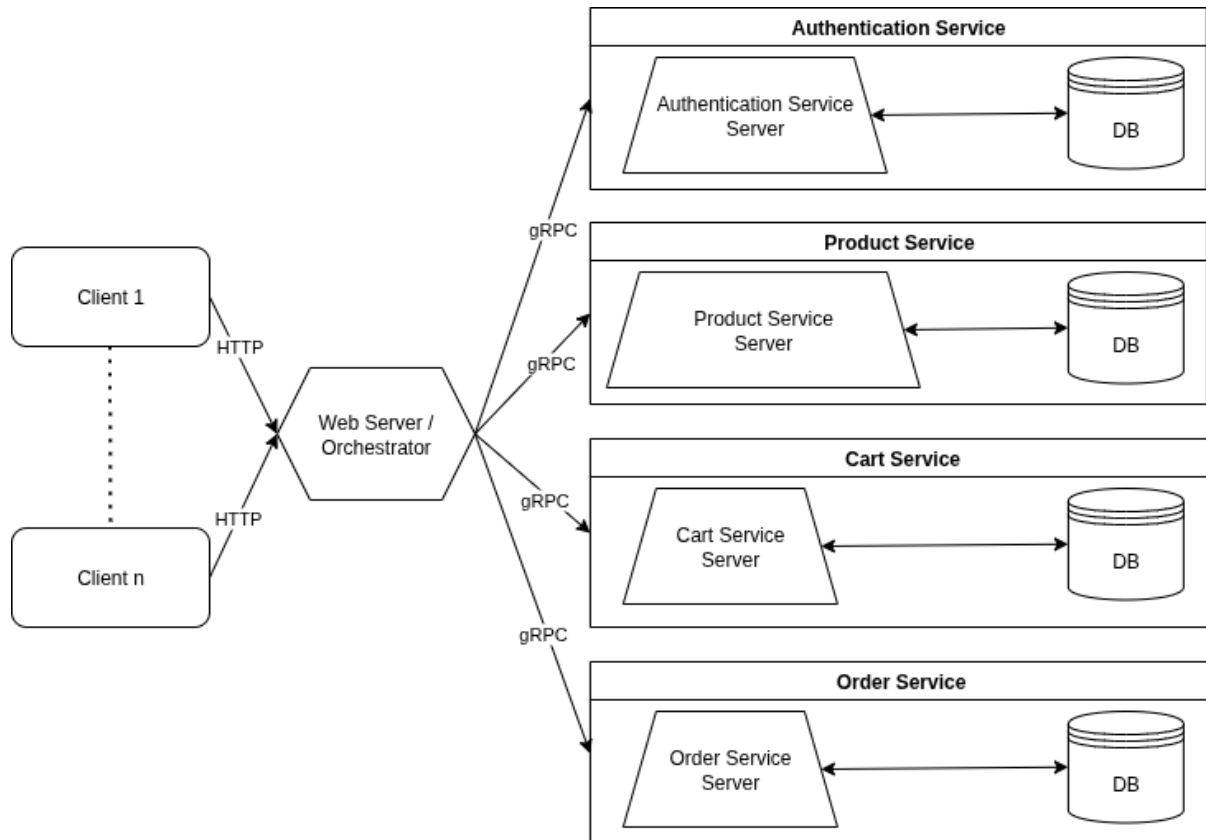


Report e-commerce

(online clothing store)



Project description

The application is an e-commerce system in which there are two types of users:

- **Client:** a user who can
 - browse the products catalogue;
 - add products to the shopping cart;
 - update quantities or remove products from the cart;
 - place orders;
 - view the list and details of its orders;
 - cancel an order (if not already canceled or delivered);
 - update its password and profile information.
- **Administrator:** a user with both admin and client privileges:
 - *Admin functions:*
 - view the list of registered users;
 - change the role of other users (client <-> administrator);
 - create, edit and delete products;
 - view the list of all orders;

- see details of each order;
- update order status (processing -> shipped -> delivered), but cannot cancel orders of other users.
- *Client functions*: same as a regular client, so admins can also purchase products.

Software Architecture

The application is an e-commerce system built on a microservices-oriented architecture.

Users (*Web Clients*) interact with the system through a web interface, which sends HTTP requests to a *Web Server*.

The Web Server forwards these requests to an *Orchestrator* responsible for handling the application logic calling methods of gRPC clients.

Each gRPC client relies on a *Service Manager* to obtain the connection to the corresponding gRPC server, that exposes business functionalities and accesses the underlying data layer.

In details, there are four different microservices:

- *Authentication*: it is responsible for authentication and user management. Provides a set of functions to enable users to register, log in, change their password and other attributes, as well as retrieve the attributes of all users.
- *Product*: it is responsible for product management. Provides a set of functions to create, retrieve, update and delete clothing products.
- *Cart*: it is responsible for shopping carts management. Provides a set of functions to
 - add a product to a cart;
 - remove a product from a cart;
 - update the quantity of an item into a cart;
 - remove all items from a cart;
 - remove a specific product from all carts;
 - list all the products, and their details, that are into a cart.
- *Order*: it is responsible for orders management. Provides a set of functions to
 - create an order;
 - update the status of an order (processing, shipped, delivered, canceled);
 - retrieve the list of all orders that have been placed by a specific user;
 - retrieve the list of all orders that have been placed by all users;
 - retrieve details for a specific order.

Main software components

Web Client

A web client establishes a connection and interacts with the Web Server through the web application interface.

Web Server

The application instantiates a Web Server, struct *HTTPWebServer*, that listens for incoming client connections and exposes an HTTP API to all clients.

The server statically binds HTTP routes to handler functions to process each request. Most of these handlers rely on the *Orchestrator*'s methods to interact with microservices.

Service Manager

The application instantiates a Service Manager, struct *ServiceManager*, responsible for maintaining reliable connections with gRPC servers. Each gRPC server creates an instance of a gRPC '*health.Server*' to enable service health monitoring.

Specifically, the Service Manager monitors each gRPC server in a separate Go routine and operates as follows:

1. The Service Manager attempts to establish a new connection for each gRPC server.
2. If the service is healthy:
 - The service status is set to ready.
 - The connection is considered valid.
 - The monitoring routine pauses for 15 seconds before the next check.
3. If the service is unhealthy:
 - The service status is set to non-ready.
 - The monitoring routine waits 2 seconds before retrying the connection.

The Service Manager exposes two methods to retrieve a connection to a server by its name: one with a timeout, which waits for the service to become ready until the timeout expires, and one without a timeout.

When the Service Manager is instantiated, a map of service names to addresses is provided to its constructor. This allows the Service Manager to be configured to monitor a wide variety of services.

Orchestrator

The Orchestrator, struct *ServiceOrchestrator*, is a core software component that acts as middleware between the Web Server and all gRPC clients. The Web Server calls Orchestrator methods to interact with the required microservices.

The Orchestrator uses the injected gRPC client instances to call the methods of the target microservice, executes the required functionality, waits for the response, and returns it to the Web Server.

The gRPC client instances are injected into the Orchestrator at the time of its instantiation.

These clients must implement the interfaces defined in the package interfaces, namely

- `interfaces.AuthClientInterface`,
- `interfaces.ProductClientInterface`,
- `interfaces.CartClientInterface`, and
- `interfaces.OrderClientInterface`.

In fact, the Orchestrator depends only on interfaces to increase the decoupling with gRPC clients without depending on their actual implementation.

gRPC Clients

Each gRPC client implements its corresponding interface, defined in the interfaces package, so that it can be injected to the Orchestrator during instantiation.

When instantiating a client, its constructor receives the Service Manager instance, allowing the client to retrieve the connection to the corresponding gRPC server when needed.

Each client has a private method *getClient()*, which initially obtains the connection to the gRPC server and then creates and returns the actual Protobuf gRPC client using the code generated from the compiled .proto files.

Thus, all methods of each client first call the *getClient()* method to obtain the Protobuf client, and then use it to invoke the service request.

gRPC Servers

Each gRPC server implements and exposes all services defined in its corresponding .proto file. Interaction with the database within each server is performed using the Repository design pattern: each server's Go module contains an interfaces package that defines the interface which the injected database must implement. This design pattern increases the decoupling between the server and the database implementation, simplifying testing through the use of mock databases.

It is noteworthy that extensive input validation is performed at the server level rather than at the database level. This ensures that the sole responsibility of the repository interface implementation is to interact with the database, without having knowledge of the data semantics.

System architecture

The system is based on the client-server paradigm, where Web Clients communicate with the Web Server to access the application functionalities.

The system follows a three-tier architecture: a presentation tier (external Web Clients such as browsers), an application tier (Web Server, Orchestrator, and microservices), and a data tier (databases accessed by the gRPC servers). The application and data tiers can be deployed independently as Docker containers, simulating separate nodes for each service. All containers share a common Docker subnet, with only the Web Server container exposing its port to the host (mapped to WEB_EXPOSE_PORT), allowing external clients to access the application functionalities.

Communication

The e-commerce system relies on two types of communication:

- **HTTP**: used for the interaction between Web Clients and the Web Server. This allows users to access the system through the web interface from a browser.
- **gRPC**: used for the communication between the Web Server (through the Orchestrator) and the microservices. This mechanism is particularly efficient for these interactions. Compared to traditional REST/HTTP communication based on JSON, gRPC transmits messages in a compact binary format (Protocol Buffers); this avoids manual marshalling and unmarshalling of data structures, reducing computational overhead and network payload size.

Technologies Adopted

This section describes the main technologies adopted for the project implementation.

Gin Web Framework

Gin is an HTTP web framework written in Go. In this project, it has been adopted because it greatly simplifies the binding of routes to handler functions, as well as the management of sessions and cookies.

Moreover, Gin provides an efficient way to control access to routes by grouping them according to authorization levels. For this purpose, the *HTTPWebServer* struct defines two helper methods:

- *AuthRequired()*: ensures that a user is authenticated before accessing certain routes. If a non-logged user attempts access, it is redirected to the login page.
- *AdminRequired()*: restricts access to administrator-only routes. If a non-admin user attempts access, a warning message is shown and the user is redirected to the main page of the website.

Thanks to these features, Gin allows for a clear and maintainable definition of route access policies, improving both security and usability of the application. In addition, Gin is known for its high performance and minimal overhead, making it particularly suitable for web applications that require fast request handling.

GORM: ORM library for Golang

GORM is an Object Relational Mapper (ORM) for Go that optimises the interaction with relational databases. In this project, the database schema was defined in advance through SQL scripts, while GORM was employed for all subsequent data access and manipulation. By mapping database tables to Go structures, GORM eliminates the need to write raw SQL queries directly in the code, improving readability and maintainability. Its high-level methods (e.g., Create, Find, Update, Updates, Where) enable developers to perform common database operations more efficiently, reducing repetitive and error-prone boilerplate code.

In the project, GORM has been used for the concrete implementation of the repository interfaces defined within each gRPC server. This approach ensures that the servers remain decoupled from the actual database technology, as they only depend on the repository interface. Thanks to this design, the database layer can be easily replaced or mocked during testing without affecting the rest of the application logic.

Further Project Details

Logger

For this project, a custom logging module has been implemented. The module github.com/marcoaga02/dp-ecommerce-project/ecommerce/logger defines a *Logger* interface and provides a concrete implementation, *StdLogger*, which outputs log messages to the standard output.

The logger supports different severity levels: Debug, Info, Warn, and Error. When a new `StdLogger` instance is created, it is initialized with a minimum logging level; only messages with a severity greater than or equal to that level are actually logged.

Each log message includes both the severity level and the component from which the message originates, providing useful context for understanding the source and criticality of the event.

In addition, the logger package includes an utility function, *ParseLogLevel()*, which translates a textual key into the corresponding *logger.Level*. If an invalid key is provided, the function defaults to the Info level.

Web Templates

The web interface of the e-commerce system is implemented using HTML templates. Within the module github.com/marcoaga02/dp-ecommerce-project/ecommerce/orchestrator, the *templates* folder contains all the HTML templates used to render the web pages.

Two of these templates, *header.html* and *footer.html*, are included respectively at the beginning and at the end of every other template. This approach ensures consistency across all pages by avoiding code duplication and guaranteeing that the header and footer remain identical throughout the application.

The *header.html* template is also responsible for displaying flash messages, which provide immediate feedback to users after specific operations. Three types of flash messages are supported:

- **Flash**: informational messages displayed with a green background after a successful operation.
- **Error**: error messages displayed with a red background, indicating an internal server issue (not caused by the user).
- **Warning**: warning messages displayed with a yellow background, indicating an unsuccessful operation due to user input errors (e.g., filling out a form incorrectly).

Test

Test cases have been implemented specifically for the repository implementations to verify that interactions with all databases work correctly. For this purpose, each db folder within every service module contains an *initTest.sql* file, which is used to initialize the test database inside the corresponding Docker container.

Unit tests are located in the *internal/test* folder of each service module. Before executing each unit test, the database is cleaned and initialized according to the specific context being tested. This ensures that all tests execute successfully and independently, without relying on a particular execution order.

Instruction to run the code and tests

The deployment of the project relies on **Docker**, **Docker Compose**, and **Makefile**. These dependencies must be installed in order to run the project.

All the following make commands can take the argument `TARGET_SERVICES` from the command line, to execute only the needed yaml file for docker compose:

- e.g. `make build TARGET_SERVICES="auth order"` will perform the build of only auth and order services.

The value that can be passed to such argument are:

- for target run (code section): *auth*, *product*, *cart*, *order*, *orchestrator*,
- for target test (test section): *auth_test*, *product_test*, *cart_test*, *order_test*.

It is particularly useful while executing tests, to see the results of tests for each single service, but not really interesting for the project execution.

Code Section

Compile and run the code

To compile and run the project, execute the following commands in order from the root folder of the project (where the Makefile is located):

1. **make build**
2. **make up** (or **make up_bg**, to execute the code in the background: logs will not be displayed)

Once the containers are running, open a browser on the machine where the project is executed and go to

- <http://localhost:3000> (or use a different port if you have configured the variable `WEB_EXPOSE_PORT` in the *deployment/.env* file with a different value)

to access the e-commerce web interface.

Note: Starting all components may take several seconds. If some required components have not yet started, you may receive an Internal Server Error. Wait a few seconds and the system will be fully operational.

Stop the execution

- If you used **make up**, to stop all containers press **Ctrl+C** in the same terminal.
- If you used **make up_bg**, to stop all containers, execute **make stop**.

Clean Up

Docker containers are configured with volumes to persist data across executions. To remove all Docker volumes and networks created for the e-commerce system, use the command **make down**.

After running **make down**, you must execute **make build** again before using **make up** to restart the project.

Test section

Compile and run all tests

To build containers, compile .proto files and run all tests, execute the following command from the root folder of the project (where the Makefile is located):

- **make run_test**

The results of all tests will be displayed in the terminal.

Clean up

To remove all Docker volumes and networks created for the e-commerce system, use the command **make down_test**.

Use cases

In this section, two different use cases are presented: one from the perspective of an administrator and one from the perspective of a client.

When starting the ecommerce, default users and products are generated. Regarding users, two default users are created:

- an administrator:
 - username: admin
 - password: admin123
- a client:
 - username: demo-client
 - password: demo123

Note: A default administrator is required because only administrators have the ability to promote a client to an administrator. Since newly registered users are assigned the client role by default, this ensures that there is at least one administrator in the system. Furthermore, an administrator cannot change its own role, preventing the system from being left without any administrators.

Client perspective

Register (new user)

Instead of using the default user, as will be done for the administrator, let's register a new client by clicking on the "Register" button from the navigation bar at the top of the screen.

Important information for a successful registration:

- password and confirmation must be exactly the same;
- use a valid mail format, e.g. [something@somethingelse.domain](#);
- use a numeric phone.

Register

Username:

Password:

Confirm Password:

Email:

Phone:

[Register](#)

Note: passwords are obscured in the e-commerce system; they are displayed here only for clarity.

Login (using the new username)

After a successful registration, you will be redirected to the login page. Here, you can access the e-commerce site using your registered username and password.

Login

Username:

Password:

[Login](#)

Note: passwords are obscured in the e-commerce system; they are displayed here only for clarity.

After the login a flash message (green) is displayed, indicating the role (here client)

Successful login! You are a client!

and the client will see the following navigation bar in the top of the screen

Hello marco[Home](#)[Shop](#)[Cart](#)[Orders](#)[Profile](#)[Change Password](#)[Logout](#)

and the following panel in the middle of the screen.

Welcome to My Ecommerce

Where you can buy clothes for every occasion

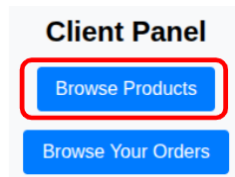
Client Panel

[Browse Products](#)

[Browse Your Orders](#)

Browse Products

After clicking on the “Browse Products” button



the list of all products will be shown (the list of the default ones, if no administrator has edited them).

List of Products

Name	Size	Color	Description	Availability	Price	Buy
Grey Classic Fleece Hoodie	S	Grey	Hoodie with front pocket	75 in stock	27.75 €	Buy
Grey Oversized Fleece Hoodie	XXL	Grey	Hoodie with front pocket	45 in stock	18.25 €	Buy
Blue Denim Large Jeans	L	Blue	Durable straight-cut denim jeans	50 in stock	49.99 €	Buy
Blue Denim Slim Jeans	S	Blue	Durable straight-cut denim jeans	50 in stock	45.99 €	Buy
White Cotton T-Shirt	M	White	Comfortable 100% cotton t-shirt for everyday wear	100 in stock	24.99 €	Buy
Black Cotton T-Shirt	XS	Black	Comfortable 100% cotton t-shirt for everyday wear	85 in stock	29.99 €	Buy
Oversized Cotton T-Shirt	XL	Black	Comfortable 100% cotton t-shirt for everyday wear	85 in stock	19.99 €	Buy

The only thing that a client can do from this page is to click on the “Buy” button of a product. Let’s click on the first one for product “Grey Classic Fleece Hoodie” and in the new page you will be able to add a certain quantity of product to the cart (up to the [availability - quantity in cart]). Let’s set the quantity at 10 units and click on the “Add to Cart” button.

Buy Product

Name	Size	Color	Description	Availability	In Cart	Price	Quantity	Add to Cart
Grey Classic Fleece Hoodie	S	Grey	Hoodie with front pocket	75 in stock	0	27.75 €	10	Add to Cart

1 2

[Back to Products](#)

Note: by clicking on the button “Back to Products” under the table, on the left side, you will be redirected to the page with the list of products.

After clicking on the “Add to Cart” button, the product is added to the user’s cart and you’ll be redirected to your shopping cart.

Successful addition of 10 products to the cart

Shopping Cart

Product	Price	Quantity	Subtotal	Actions
Grey Classic Fleece Hoodie	27.75 €	10	277.50 €	Modify Remove

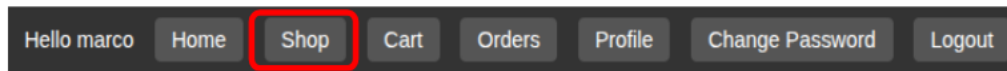
Total: 277.50 €

[Back to Products](#)

[Clear Cart](#)

[Place order](#)

To go again to the page with all the products, you can click on the “Back to Products” button or you can click on the “Shop” button in the navigation bar.



Let's add two other products and the shopping cart will contain those products.

Shopping Cart

Product	Price	Quantity	Subtotal	Actions	
Grey Classic Fleece Hoodie	27.75 €	10	277.50 €	Modify	Remove
Blue Denim Large Jeans	49.99 €	4	199.96 €	Modify	Remove
Black Cotton T-Shirt	29.99 €	13	389.87 €	Modify	Remove

Total: 867.33 €

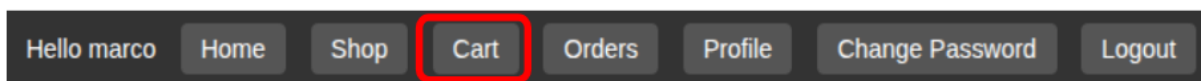
[Back to Products](#)

[Clear Cart](#)

[Place order](#)

Cart

Clicking on the “Cart” button in the navigation bar,



you will see the products that are into your cart, or a warning message if the cart is empty.

Shopping Cart

Product	Price	Quantity	Subtotal	Actions	
Grey Classic Fleece Hoodie	27.75 €	10	277.50 €	Modify	Remove
Blue Denim Large Jeans	49.99 €	4	199.96 €	Modify	Remove
Black Cotton T-Shirt	29.99 €	13	389.87 €	Modify	Remove

Total: 867.33 €

[Back to Products](#)

[Clear Cart](#)

[Place order](#)

Here, you can do several things (if at least one product is into the cart):

- remove all the products from the cart by clicking on the “Clear Cart” button;
- remove one or more products by clicking to the corresponding “Remove” button and confirming the choice in the panel;
- edit the quantity of a product into the cart:
 - a. click on the modify button to enable changes to the quantity

Product	Price	Quantity	Subtotal	Actions		
Grey Classic Fleece Hoodie	27.75 €	10	277.50 €	Save	Cancel	Remove

- b. update the quantity
 - can be set to zero, and the product will be removed from the cart
 - cannot be greater than the stock
- c. click on the “Cancel” button to not save the changes, or click on the “Save” button to save changes. Let's change the quantity to 15, click save and you will see the updated quantity.

- place the order by clicking on the “Place Order” button

Shopping Cart

Product	Price	Quantity	Subtotal	Actions
Grey Classic Fleece Hoodie	27.75 €	15	416.25 €	Modify Remove
Blue Denim Large Jeans	49.99 €	4	199.96 €	Modify Remove
Black Cotton T-Shirt	29.99 €	13	389.87 €	Modify Remove

Total: 1006.08 €

[Back to Products](#)

[Clear Cart](#)

[Place order](#)

After clicking on this button, the order is added to the orders list, the availability of those products is decreased and you are redirected to the orders page.

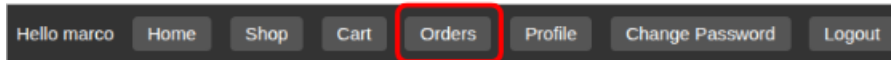
Successful creation of the order with ID '1'

Orders

Order ID	Status	Total Amount (€)	Actions
1	processing	1006.08	View

Orders

By clicking on the button “Orders” in the navigation bar,



you will be redirected to the orders page in which the list of the user orders is displayed. If you have created an order, as shown at the end of the previous section, you will see something like this

Orders

Order ID	Status	Total Amount (€)	Actions
1	processing	1006.08	View

Each order has a “View” button: by clicking on it, you will see all details of the order and its status (processing, shipped, delivered, canceled).

Order Details

Order ID: 1

Status: processing

Total Amount: 1006.08 €

Items

Product Code	Name	Unit Price (€)	Quantity	Total (€)
HOODIE-001	Grey Classic Fleece Hoodie	27.75	15	416.25
JEANS-001	Blue Denim Large Jeans	49.99	4	199.96
TSHIRT-002	Black Cotton T-Shirt	29.99	13	389.87

[Cancel Order](#)

If you want to cancel the order, you can click on the “Cancel Order” button and the status of the order will become “canceled”. If the order has been already delivered / canceled, the cancel button disappears. The update of the status from “processing to shipped” and from “shipped to delivered” can be performed only by an administrator.

Order successfully canceled

Order Details

Order ID: 1

Status: **canceled** ← note the status updated to canceled

Total Amount: 1006.08 €

Items

Product Code	Name	Unit Price (€)	Quantity	Total (€)
HOODIE-001	Grey Classic Fleece Hoodie	27.75	15	416.25
JEANS-001	Blue Denim Large Jeans	49.99	4	199.96
TSHIRT-002	Black Cotton T-Shirt	29.99	13	389.87

← The “Cancel Order” button is not present after the cancellation of the order

After the cancellation of an order, the availability of the products that were part of the order is increased again.

Now, let’s create another order that will be useful to show the power of an administrator later.

Successful creation of the order with ID '2'

Orders

Order ID	Status	Total Amount (€)	Actions
2	processing	759.80	View
1	canceled	1006.08	View

User Profile

A user can see its profile by clicking on the “Profile” button in the navigation bar

Hello marco Home Shop Cart Orders **Profile** Change Password Logout

and its information will be displayed

User Profile

Username: marco

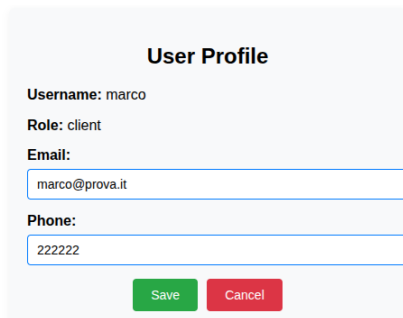
Role: client

Email:

Phone:

[Modify](#)

In the case in which either the email or the phone number is wrong, or both, it is possible to modify them by clicking on the “Modify” button, to enable the editing.



User Profile

Username: marco

Role: client

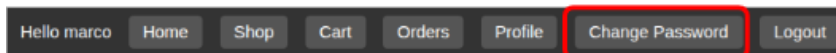
Email:

Phone:

After all changes have been performed, it is possible to click on the “Save” button, to confirm these changes, or on the “Cancel” one, to not save them.

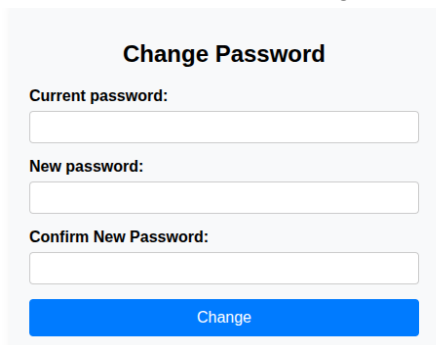
Change Password

A user can change its password by clicking on the “Change Password” button.



Hello marco

After the click, the following form will be displayed



Change Password

Current password:

New password:

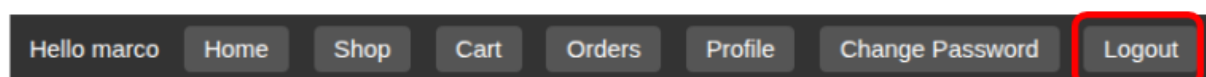
Confirm New Password:

and the user can change its password by firstly writing the current password and the new one and finally clicking on the “Change” button.

In the case of a successful password change, the user will be redirected to the main page of the ecommerce site.

Logout

A user can log out of the ecommerce site by clicking on the “Logout” button in the navigation bar and it will be redirected to the login page.

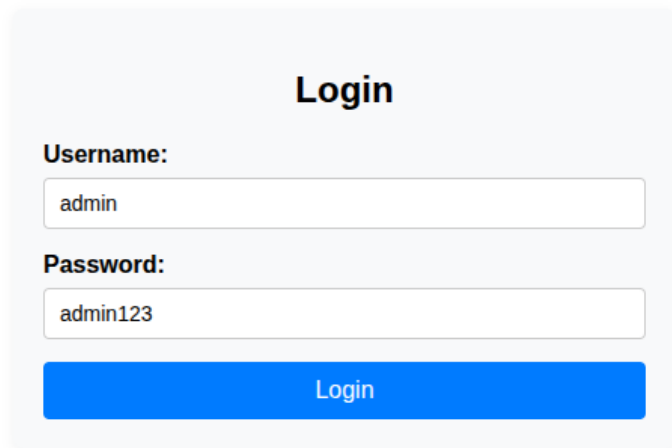


Hello marco

Administrator perspective

Login (default administrator)

Firstly, log in into the website as an administrator by using the default administrator with username “admin” and password “admin123”



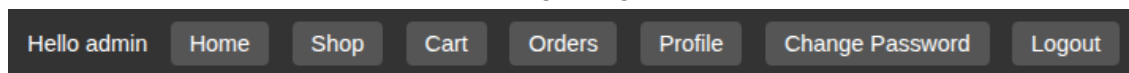
The login form is titled "Login" in bold black text. It contains two input fields: "Username:" with the value "admin" and "Password:" with the value "admin123". Below the password field is a blue "Login" button.

Note: password is obscured in the e-commerce system; it is displayed here only for clarity.

After the login a flash message (green) is displayed, indicating the role (here administrator),

Successful login! You are an administrator!

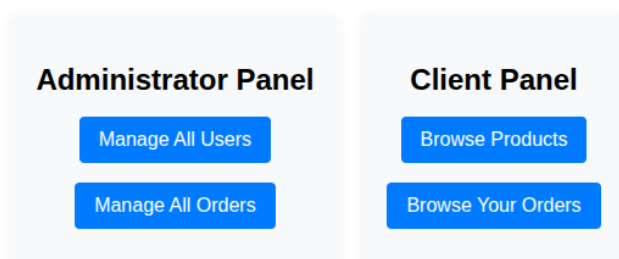
and the administrator will see the following navigation bar in the top of the screen



and two panels in the middle of the screen: the first panel is dedicated to administrative functions, while the second panel is intended for client actions but the administrator has more power, as explained later.

Welcome to My Ecommerce

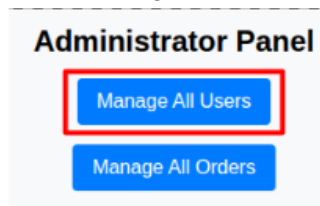
Where you can buy clothes for every occasion



In this section, only administrative functions will be shown, since the client ones have been shown in the previous section (they are the same apart from some additional information that an administrator sees).

Manage All Users

After clicking on the “Manage All Users” button



all users but the “caller” will be shown, and their role, one by one, can be updated by selecting the desired role and finally clicking on the “Save” button.

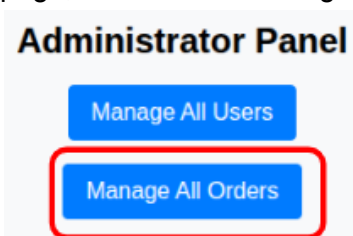
Manage Users

Username	Email	Phone	Role	Action
demo-client	demo-client@example.com	1111111111	Client Client Administrator	Save
marco	marco@prova.it	222222		Save

If the operation is successful, a green flash message will be displayed.

Manage All Orders

Now, let’s click on the “Home” button from the navigation bar and, after reaching the home page, click on the “Manage All Orders” button.



If you have placed the two orders from the client, as done in the previous section, now you will see a table with two rows.

Orders

Order ID	Status	Total Amount (€)	Actions
2	processing	759.80	View
1	canceled	1006.08	View

Since the order with ID 1 has been canceled, it is no longer interesting to show the functionality of the administrator. So, let’s click on the “View” button of the order with ID 2 and the details of the order will be displayed.

Order Details

Order ID: 2

Status: processing

Total Amount: 759.80 €

Items

Product Code	Name	Unit Price (€)	Quantity	Total (€)
JEANS-001	Blue Denim Large Jeans	49.99	8	399.92
TSHIRT-002	Black Cotton T-Shirt	29.99	12	359.88

Update Status: Processing

Update Status

The status is set to “processing” and the administrator can change it to “shipped”, by selecting “Shipped” from the menu under the table and then clicking on the “Update Status” button.

Items

Product Code	Name	Unit Price (€)	Quantity	Total (€)
JEANS-001	Blue Denim Large Jeans	49.99	8	399.92
TSHIRT-002	Black Cotton T-Shirt	29.99	12	359.88

Update Status:

Shipped
Processing
Shipped

Update Status

2

1

After the update, the new status “shipped” is displayed in the order’s description and the administrator can now update the status from “shipped” to “delivered” using the same mechanism. After that, the new status will become “delivered” and the “Update Status” section disappears since after delivery, the status cannot be updated further.

Browse Products

If the administrator clicks on the “Browse Products” button from the panel in the main page,

Welcome to My Ecommerce

Where you can buy clothes for every occasion

Administrator Panel

Manage All Users

Manage All Orders

Client Panel

Browse Products

Browse Your Orders

or equivalently clicks on the “Shop” button in the navigation bar, the list of all available products will be displayed.

List of Products

Add New Product

Code	Name	Size	Color	Description	Availability	Price	Actions	Buy
HOODIE-001	Grey Classic Fleece Hoodie	S	Grey	Hoodie with front pocket	75 in stock	27.75 €	<button>Edit</button> <button>Delete</button>	<button>Buy</button>
HOODIE-002	Grey Oversized Fleece Hoodie	XXL	Grey	Hoodie with front pocket	45 in stock	18.25 €	<button>Edit</button> <button>Delete</button>	<button>Buy</button>
JEANS-001	Blue Denim Large Jeans	L	Blue	Durable straight-cut denim jeans	42 in stock	49.99 €	<button>Edit</button> <button>Delete</button>	<button>Buy</button>
JEANS-002	Blue Denim Slim Jeans	S	Blue	Durable straight-cut denim jeans	50 in stock	45.99 €	<button>Edit</button> <button>Delete</button>	<button>Buy</button>
TSHIRT-001	White Cotton T-Shirt	M	White	Comfortable 100% cotton t-shirt for everyday wear	100 in stock	24.99 €	<button>Edit</button> <button>Delete</button>	<button>Buy</button>
TSHIRT-002	Black Cotton T-Shirt	XS	Black	Comfortable 100% cotton t-shirt for everyday wear	73 in stock	29.99 €	<button>Edit</button> <button>Delete</button>	<button>Buy</button>
TSHIRT-003	Oversized Cotton T-Shirt	XL	Black	Comfortable 100% cotton t-shirt for everyday wear	85 in stock	19.99 €	<button>Edit</button> <button>Delete</button>	<button>Buy</button>

In this page, with respect to the client’s view, an administrator additionally sees:

- “Add New Product” button;
- “Edit” buttons;
- “Delete” buttons;
- Product codes.

Add New Product

After clicking on the “Add New Product” button, the following form will be displayed.

Add New Product

Code:

Name:

Size:

-- Select size --

Color:

Description:

Availability (stock):

Price (€):

Create Product

Cancel

If the user clicks on the “Cancel” button, it is redirected to the products page. Otherwise, the administrator can fill up the form and by clicking on the “Create Product” button the product will be added to the products list and the user redirected to the products page. For example, let’s create the following product by filling up the form and then clicking on the “Create Product” button

Add New Product

Code:

Name:

Size:

Color:

Description:

Availability (stock):

Price (€):

Create Product

Cancel

After the successful creation of the product, the user is redirected to the products page, a flash message (green) is displayed as well as the new product in the table.

Product with code 'TRIAL-001' successfully created

List of Products

Add New Product

Code	Name	Size	Color	Description	Availability	Price	Actions		Buy
HOODIE-001	Grey Classic Fleece Hoodie	S	Grey	Hoodie with front pocket	75 in stock	27.75 €	Edit	Delete	Buy
HOODIE-002	Grey Oversized Fleece Hoodie	XXL	Grey	Hoodie with front pocket	45 in stock	18.25 €	Edit	Delete	Buy
JEANS-001	Blue Denim Large Jeans	L	Blue	Durable straight-cut denim jeans	42 in stock	49.99 €	Edit	Delete	Buy
JEANS-002	Blue Denim Slim Jeans	S	Blue	Durable straight-cut denim jeans	50 in stock	45.99 €	Edit	Delete	Buy
TRIAL-001	Trial t-shirt	XXL	Green	A trial green t-shirt	23 in stock	11.29 €	Edit	Delete	Buy
TSHIRT-001	White Cotton T-Shirt	M	White	Comfortable 100% cotton t-shirt for everyday wear	100 in stock	24.99 €	Edit	Delete	Buy
TSHIRT-002	Black Cotton T-Shirt	XS	Black	Comfortable 100% cotton t-shirt for everyday wear	73 in stock	29.99 €	Edit	Delete	Buy
TSHIRT-003	Oversized Cotton T-Shirt	XL	Black	Comfortable 100% cotton t-shirt for everyday wear	85 in stock	19.99 €	Edit	Delete	Buy

Delete

If an administrator wants to delete a product, it is sufficient to click on the “Delete” button corresponding to that product from the products table.

After deletion, all the carts containing the product will be updated, by removing it from them.

Edit

If an administrator wants to edit a product, it is sufficient to click on the “Edit” button corresponding to that product from the products table.

Let's modify the first product with code “HOODIE-001”, by clicking on its “Edit” button.

List of Products

[Add New Product](#)

Code	Name	Size	Color	Description	Availability	Price	Actions	Buy
HOODIE-001	Grey Classic Fleece Hoodie	S	Grey	Hoodie with front pocket	75 in stock	27.75 €	Edit Delete	Buy
HOODIE-002	Grey Oversized Fleece Hoodie	XXL	Grey	Hoodie with front pocket	45 in stock	18.25 €	Edit Delete	Buy
JEANS-001	Blue Denim Large Jeans	L	Blue	Durable straight-cut denim jeans	42 in stock	49.99 €	Edit Delete	Buy
JEANS-002	Blue Denim Slim Jeans	S	Blue	Durable straight-cut denim jeans	50 in stock	45.99 €	Edit Delete	Buy
TRIAL-001	Trial t-shirt	XXL	Green	A trial green t-shirt	23 in stock	11.29 €	Edit Delete	Buy
TSHIRT-001	White Cotton T-Shirt	M	White	Comfortable 100% cotton t-shirt for everyday wear	100 in stock	24.99 €	Edit Delete	Buy
TSHIRT-002	Black Cotton T-Shirt	XS	Black	Comfortable 100% cotton t-shirt for everyday wear	73 in stock	29.99 €	Edit Delete	Buy
TSHIRT-003	Oversized Cotton T-Shirt	XL	Black	Comfortable 100% cotton t-shirt for everyday wear	85 in stock	19.99 €	Edit Delete	Buy

The following form will be displayed and by modifying its fields and by clicking on the “Update Product” button, all changes will be performed.

Edit Product

Code:
HOODIE-001

Name:

Size:

Color:

Description:

Availability (stock):

Price (€):

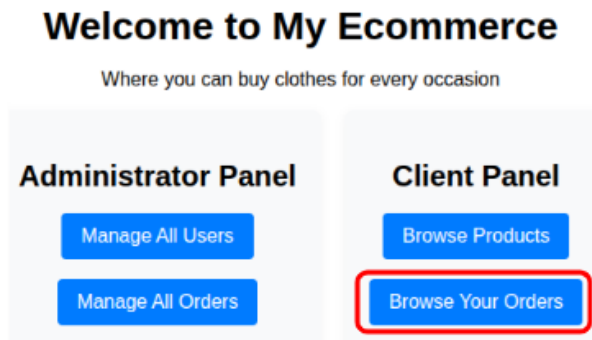
[Update Product](#)

[Cancel](#)

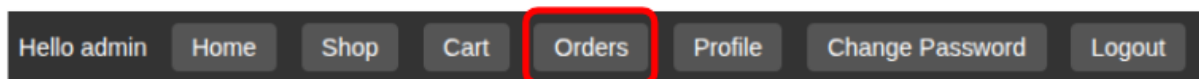
If the administrator wants to cancel all changes, before saving them, it can click on the “Cancel” button and it will be redirected to the products page after the confirmation of its choice.

Browse Your Orders

By clicking on the “Browse Your Orders” button in the main page



or by clicking on the “Orders” button from the navigation bar



the administrator will see its orders, exactly as a client.

Let's place an order for the administrator and the orders page will show something like this

Orders

Order ID	Status	Total Amount (€)	Actions
3	processing	123.24	View

If the administrator clicks on the “View” button, it will be displayed the following page.

Order Details

Order ID: 3

Status: processing

Total Amount: 123.24 €

Items

Product Code	Name	Unit Price (€)	Quantity	Total (€)
HOODIE-001	Grey Classic Fleece Hoodie	27.75	2	55.50
TRIAL-001	Trial t-shirt	11.29	6	67.74



In this page, with respect to the client's view, an administrator additionally sees the “Update Status” section. In fact, it can cancel its order, as a regular client, but additionally it can also update the status of the order, as a regular administrator.